Université Abou Bekr Belkaid

Faculté des Sciences – Tidjani Haddam -

Département d'informatique

جامعة أبو بكر بلقايد
UNIVERSITÉ DE TLEMCEN

TP1 HPC 2026/2027

4 Ingénieur IA

Table des matières

1. Premier programme MPI
2. Activité 1: Communication point à point
3. Activité 2: Communication collective
4. Activité 3: Mesure du temps d'exécution

1.Premier programme MPI:

➢ Création du fichier source

1. Ouvre un terminal
2. Crée un fichier nommé **hellompi.c (vi /nano …)** dans le dossier cloud
3. Copie le code suivant dans le fichier :

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char *argv[]) {
   int rank, size, len;
   char processor_name[MPI_MAX_PROCESSOR_NAME];
   MPI_Init(&argc, &argv); // Initialisation de l'environnement MPI
   MPI_Comm_size(MPI_COMM_WORLD, &size); // Nombre total de processus
   MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Rang du processus (identifiant)
   // Récupération du nom du nœud (machine)
   MPI_Get_processor_name(processor_name, &len);
   printf("Bonjour depuis le processus %d sur %d, exécuté sur le nœud : %s\n",
        rank, size, processor_name);
   MPI_Finalize(); // Finalisation de l'environnement MPI
   return 0;
}
```

➢ Compilation du programme

MPI ne se compile pas avec gcc, mais avec le compilateur MPI mpicc.

```
mpicc hellompi.c -o hellompi
```

- **Exécution du programme MPI**

Ton programme hellompi est maintenant compilé.

Tu peux le lancer avec plusieurs processus MPI pour observer le comportement parallèle.

Lancer le programme avec plusieurs processus :

1. Exécution sur un seul nœud (même machine)

   mpirun -n 4 ./hellompi

2. Exécution sur plusieurs nœuds

   mpirun -n 4 --hosts node1,node2 ./hellompi
   mpirun -n 4 --hosts node1:1,node2:3 ./hellompi
   mpirun -n 4 --hosts masteria:1,node1:1,node2:2 ./hellompi

2.Activité 1 : Communication point à point

Écrivez un programme dans lequel le processus zéro lit une donnée insérée par l'utilisateur et la transmet à tous les autres processus en l'envoyant dans un anneau. Autrement dit, le processus i devrait recevoir la donnée et l'envoyer au processus i + 1, jusqu'à ce que le dernier processus soit atteint.

Remarque : Utiliser les fonctions

```
MPI_Send(&value, 1, MPI_INT, rank, 0, MPI_COMM_WORLD);
MPI_Recv(&value, 1, MPI_INT, rank, 0, MPI_COMM_WORLD, &status);
```

3.Activité 2 : Communication collective

- Écrire un programme MPI pour répartir les lignes d'une matrice 4×4 entre 4 processus.

Le processus de rang 1 envoie une ligne à chaque processus.

Chaque processus affiche les valeurs reçues.

Si le nombre de processus est différent de 4, le programme se termine avec un message d'erreur.

- On va garder la même logique de distribution de lignes (MPI_Scatter) et ajouter une réduction (MPI_Reduce) pour calculer la somme de toutes les valeurs distribuées.

Remarque : Utiliser

```
int MPI_Scatter (void *sendbuf, int sendcount, MPI_Datatype
sendtype,
             void *recvbuf,int recvcount, MPI_Datatype recvtype,
             int root,        MPI_Comm comm)

int MPI_Reduce  (const void *sendbuf, void *recvbuf, int count,
                 MPI_Datatype datatype, MPI_Op op, int root,
                 MPI_Comm comm)
```

4.Activité 3 : Mesure du temps d'exécution

Écrire ou modifier un programme MPI permettant de mesurer le temps d'exécution de l'activité précédente (activité 2)

La communication point à point (avec MPI_Send et MPI_Recv) ;

Remarque : Utiliser

- MPI_Wtime() pour calculer le temps écoulé
- MPI_Barrier(MPI_COMM_WORLD) pour synchroniser tous les processus